Senior Software Engineer interview preparation

3 MAIN SECTIONS • PRINT-READY EDITION

Current round: technical discussion and coding. **Available study time:** 20 focused hours. **Scope:** backend technologies. Full system design belongs to your next round.

This guide is tailored to the four resumes you provided. The practice questions are preparation recommendations, not verified American Express interview questions. Your specific team, job description, Java version, and coding platform are still unknown. Use Java for practice unless the interviewer specifies otherwise.

1. Start here

Your strongest interview material connects Java and APIs to real business problems: commercial user management, payments, reconciliation, IAM/MFA, certificate automation, Redis caching, and production reliability. Your main preparation task is to turn broad leadership statements into precise explanations of implementation, tests, failures, and personal contribution.

For each concept, follow this cycle: **understand it in simple English → explain it without notes → implement or trace a small example → answer a failure question.** Reading alone is not enough for a coding round.

The professional answers in this guide are speaking templates. Learn the meaning and structure rather than every word. Adapt first-person statements to your actual experience. A clear 40-second answer is better than a memorized two-minute speech.

Resume comparison and priorities

Resume	Useful preparation evidence	How to use it
Sree.pdf	Java/J2EE, Spring MVC/Data/Integration, Vert.x, SOAP/REST, DB2/Oracle/MySQL, Couchbase, EDI and batch migration	Use for technical history and integration examples. It has an older experience summary.
Sree-Resume.pdf	Java/Spring/Vert.x, Redis, IAM/MFA, certificate automation, deployment practices; Airflow and Spark in skills	Use for backend delivery and security stories. Confirm depth behind skills-only entries.

Resume	Useful preparation evidence	How to use it
sree_resume-1.pdf	Most explicit backend detail: Vert.x/Java services, Spring Boot, Redis, Jaeger, Splunk, DR, Kubernetes/AWS, authorization	Use as the main study map. Verify each claim before repeating it.
sree_resume-2.pdf	Product strategy, payments, migration, operational outcomes	Use for business context. Add implementation details when answering engineering questions.

Priority A - demonstrate confidently: Java collections and correctness, coding patterns, SQL, concurrency, Spring transactions, Vert.x event loops, REST, authorization, and your own projects.

Priority B - explain and troubleshoot: Redis, JVM performance, testing, tracing, deployment rollback, payment retry safety, certificate rotation, Couchbase.

Priority C - brief review only: AWS/Kubernetes specifics, Node.js, Airflow/Spark, SOAP/EDI tooling, deep distributed-system architecture. Move a topic up only if it is central to the role or to a project you choose to discuss.

2. Your 20-hour plan

The hours below are focused study time. Add short breaks, meals, and a full night's sleep separately. Target hours 1-10 on Thursday and hours 11-20 on Friday; move blocks to match your actual availability. Finish demanding practice at least 60 minutes before the interview. Use the last half-hour of study for recall, not new material.

Study hours	Duration	What to do	Completion checkpoint
1	1 h	Baseline: 15 min resume/intro; 30 min Two Sum + one SQL query; 15 min review	Identify your three weakest areas; record a 60-second introduction
2-3	2 h	Java fundamentals, collections, equality, exceptions, generics, streams, money	Explain cards 1-9; code grouping and sorting without reference
4-6	3 h	Coding patterns: hashing, sliding window, intervals, binary search, stack, heap/ tree basics	Solve exercises 1-6; narrate complexity and edge cases
7-8	2 h	Concurrency, JVM, asynchronous execution, Vert.x	Explain cards 10-17; diagnose a race and a blocked event loop

Study hours	Duration	What to do	Completion checkpoint
9-10	2 h	Spring/Boot, transactions, persistence, REST and tests	Trace one request; explain rollback and self-invocation; write tests
11-12	2 h	SQL, indexes, locking, Couchbase and Redis	Write four SQL queries; explain lost update and cache invalidation
13	1 h	API security, IAM/MFA, certificate lifecycle, retry safety	Explain cards 29-35 and answer duplicate-payment scenario
14	1 h	Observability, incident response, deployment and secondary skills	Diagnose one slow API; rehearse safe rollout and rollback
15	1 h	Three project stories with technical follow-ups	Deliver each in 2 min; answer implementation, tradeoff, test and metric questions
16-17	2 h	Two timed coding sessions, including tests and review	Complete a fresh solution, explain correctness, fix one weakness
18	1 h	Full technical mock using the script below	Score yourself; write the three most important corrections
19	1 h	Repair the weakest area; repeat a failed problem from a blank editor	Solve independently without memorizing the old solution
20	1 h	30 min final verbal drill + 30 min final recall sheet	State key answers simply and consistently; stop studying

If time shrinks to 10 hours: Java/coding 4 h; concurrency/Vert.x 1 h; Spring/REST 1 h; SQL/cache 1 h; security and project stories 1 h; timed mock and correction 2 h. Skip Priority C and full system design.

If your baseline is weak: borrow up to 2 hours from secondary technologies and broad production review for Java coding. Keep SQL, transaction correctness, authorization, and your project stories.

How to use each one-hour block

Spend about 15 minutes learning, 30 minutes coding or answering aloud, and 15 minutes checking mistakes. For a concept, pass only when you can give its definition, a small example, and a limitation without reading. For code, pass only when you can explain correctness and test edge cases.

Speaking practice method

Record an answer once. Listen only for three things: Did I answer the question in the first sentence? Did I give a concrete example? Did I stop after the important tradeoff? Repeat the answer in fewer words. Do not try to remove your accent.

3. Timed mock interviews and scoring

Two coding sessions for hours 16-17

Session 1, 60 min: 5 min clarify an unfamiliar variation of sliding window or intervals; 30 min solve; 10 min tests/complexity; 15 min review and rewrite the faulty part. Example variation: find the longest substring containing at most K distinct characters.

Session 2, 60 min: 5 min clarify payment aggregation; 25 min implement; 10 min SQL latest-per-account query; 10 min race-condition or transaction debugging; 10 min review. Include mixed currencies and duplicate records in the discussion.

Full technical mock for hour 18

Minutes	Prompt	What a strong answer demonstrates
0-5	Introduce yourself and describe a recent backend contribution	Clear personal ownership and relevant technical depth
5-12	HashMap equality; volatile versus atomic increment; one stream operation	Correct mechanism and a small example
12-37	Solve merge intervals or a fresh hashing problem	Clarification, correct code, invariant, tests, complexity
37-45	SQL: latest successful payment per account	Correct filtering, partitioning, tie handling and tenant scope
45-53	A Vert.x API slows under load; retries create duplicate requests	Evidence-based diagnosis, event-loop care, safe retry semantics
53-58	Explain one project decision and a failure you handled	Truthful ownership, tradeoff, validation and learning
58-60	Ask the interviewer two questions	Interest in the team's actual engineering work

Score yourself out of 20

Give 0-4 points each for **coding correctness, complexity/reasoning, edge-case tests, technical accuracy, and clear communication/ownership**. A 0 means missing; 2 means partly correct with significant prompting; 4 means independent and precise. This is a personal practice rubric, not American Express's hiring rubric.

Target 15+ with no zero, then repair the weakest category. A confident wrong explanation is a bigger problem than a short pause followed by a correct answer.